APS — Testes Automatizados, Adequação e Análise de Mutação

João Pedro

02/09/2026

Relatório — APS: Testes Automatizados, Adequação e Análise de Mutação

Sistema sob teste: Scholarship Eligibility Evaluator (versão Python) **Linguagem / Framework / Ferramenta de mutação:** Python 3.11 · pytest 9.1 · mutmut 3.7

1. Escolha tecnológica e justificativa

O sistema já foi disponibilizado em Python, então a escolha da linguagem foi natural — evita reescrever/traduzir a lógica, o que o enunciado proíbe.

pytest foi escolhido como framework de teste por ser o padrão de fato do ecossistema Python: sintaxe simples baseada em `assert`, ótimo suporte a `pytest.raises` para testar exceções (usado nos casos de entrada inválida) e boa integração com ferramentas de mutação e cobertura.

mutmut foi escolhido como ferramenta de mutação (opção sugerida no próprio enunciado) por ser leve, não exigir infraestrutura extra (ao contrário do Cosmic Ray, que depende de um banco de sessão SQLite e arquivo de configuração TOML mais verboso) e por gerar mutantes automaticamente a partir da AST/CST do código, sem qualquer anotação manual. A versão usada (3.7) roda cada mutante fazendo uma cópia isolada do projeto em `mutants/` e reexecutando a suíte pytest contra ela.

2. Projeto dos testes — técnicas funcionais

2.1 Classes de equivalência

Critério	Classes de equivalência	Resultado esperado
Idade	EC-A1: `age < 16` · EC-A2: `16 ≤ age ≤ 17` · EC-A3: `age ≥ 18`	REJECTED · MANUAL_REVIEW · sem efeito
GPA (válido)	EC-G1: `6.0 ≤ gpa`... na verdade `gpa < 6.0` · EC-G2: `6.0 ≤ gpa < 7.0` · EC-G3: `gpa ≥ 7.0`	REJECTED · MANUAL_REVIEW · sem efeito
GPA (validação)	EC-Ginv: `gpa < 0.0` ou `gpa > 10.0`	`ValueError`
Frequência (válida)	EC-T1: `attendance < 75.0` · EC-T2: `75.0 ≤ attendance < 80.0` · EC-T3: `attendance ≥ 80.0`	REJECTED · MANUAL_REVIEW · sem efeito

Critério	Classes de equivalência	Resultado esperado
Frequência (validação)	EC-Tinv: `attendance < 0.0` ou `attendance > 100.0`	`ValueError`
Cursos obrigatórios	EC-C1: `False` · EC-C2: `True`	REJECTED · sem efeito
Histórico disciplinar	EC-D1: `True` · EC-D2: `False`	REJECTED · sem efeito
Combinação final	todas as classes "sem efeito" · alguma classe de rejeição · nenhuma rejeição mas alguma revisão · rejeição **e** revisão simultâneas	APPROVED · REJECTED · MANUAL_REVIEW · REJECTED (a revisão é mascarada)

A última linha é a classe de equivalência mais sutil do sistema: quando pelo menos um motivo de rejeição existe, o código descarta silenciosamente qualquer motivo de revisão manual acumulado (`if rejection_reasons: return REJECTED(...)`, antes de checar `review_reasons`). Ela é coberta por `test_review_reason_is_masked_when_a_rejection_also_occurs`.

2.2 Valores de fronteira

Critério	Fronteiras testadas
Idade	15/16 (rejeição→revisão) e 17/18 (revisão→aprovação)
GPA	5.9/6.0 e 6.9/7.0 (fronteiras de decisão) · 0.0 e 10.0 (extremos válidos) · -0.1 e 10.1 (primeiros valores inválidos)
Frequência	74.9/75.0 e 79.9/80.0 (fronteiras de decisão) · 0.0 e 100.0 (extremos válidos) · -0.1 e 100.1 (primeiros valores inválidos)

No total, a suíte final exercita 12 pares de fronteira (muito acima do mínimo de 4 exigido).

3. Projeto dos testes — técnicas estruturais

O código tem 8 decisões relevantes (`if`/`elif`) dentro de `evaluate_scholarship` e 2 dentro de `_validate_inputs`, além da composição final (`if rejection_reasons` / `if review_reasons` / `else`). Cada ramo foi mapeado a pelo menos um teste:

Decisão no código	Ramo	Teste(s) que exercitam
`if age < 16`	True / False	`test_age_below_16_is_rejected`, `test_age_boundary_15_16_*`
`elif age <= 17`	True / False	`test_age_16_and_17_both_trigger_manual` `test_age_boundary_17_18_*`
`if gpa < 6.0`	True / False	`test_gpa_below_minimum_is_rejected`, `test_gpa_boundary_5_9_6_0_*`
`elif gpa < 7.0`	True / False	`test_gpa_manual_review_range`, `test_gpa_boundary_6_9_7_0_*`
`if attendance_rate < 75.0`	True / False	`test_attendance_below_minimum_is_rejec`
`elif attendance_rate < 80.0`	True / False	`test_attendance_manual_review_range`
`if not has_required_courses`	True / False	`test_missing_required_courses_is_rejec` / `test_has_required_courses_true_does_no`

Decisão no código	Ramo	Teste(s) que exercitam
`if disciplinary_record`	True / False	`test_disciplinary_record_is_rejected` / `test_no_disciplinary_record_does_not_r`
`if gpa < 0.0 or gpa > 10.0` (validação)	ambos os lados do `or`	`test_gpa_below_zero_raises_value_error` `test_gpa_above_ten_raises_value_error`
`if attendance < 0.0 or attendance > 100.0`	ambos os lados do `or`	`test_attendance_below_zero_raises_valu` `test_attendance_above_hundred_raises_v`
Decisão final (rejeição / revisão / aprovação)	3 caminhos	`test_all_five_rules_trigger_rejection_` `test_manual_review_with_three_simultan` `test_approved_when_all_criteria_are_me`
Máscara rejeição-sobre-revisão	caminho combinado	`test_review_reason_is_masked_when_a_re`

`test_all_five_rules_trigger_rejection_in_declared_order` e `test_courses_and_disciplinary_combined_r` também servem como testes de **caminho** (combinações de decisões), não apenas de branch isolado, e verificam a ORDEM exata da lista `reasons`.

Todos os 29 testes verificam `status` **e** `reasons` (ou a mensagem exata da exceção, nos casos inválidos) — nunca apenas o status, conforme exigido.

4. Execução

```
pytest        # 29 passed
```

Reprodutível em qualquer máquina com Python 3.10+ e `pip install -r requirements.txt` (ver README.md para detalhes).

5. Análise de mutação

5.1 Score inicial

```
mutmut run
```

resultou em **82/84 mutantes mortos → score inicial = 97,62 %** (`mutation_reports/initial/summary.json`). O mutmut, nesta versão, só gera mutantes dentro de corpos de função — por isso não apareceu nenhum mutante no bloco de exemplo `if __name__ == "__main__":` nem no decorator `@dataclass(frozen=True)` (ambos em nível de módulo/classe). Ou seja, o "ruído" que o enunciado avisa (mutantes em blocos de exemplo de uso) simplesmente não ocorreu nesta execução — não porque o descartamos, mas porque a ferramenta, por padrão, já não os gera. Registramos isso explicitamente para não parecer uma omissão.

5.2 Mutantes sobreviventes analisados

Mutante 1 — `_validate_inputs__mutmut_7` (lacuna da suíte)

```
- raise ValueError("GPA must be between 0 and 10.")
+ raise ValueError("XXGPA must be between 0 and 10.XX")
```

Mutante 2 — `_validate_inputs__mutmut_16` (lacuna da suíte)

```
- raise ValueError("Attendance rate must be between 0 and 100.")
+ raise ValueError("XXAttendance rate must be between 0 and 100.XX")
```

Os dois mudam claramente o comportamento observável (a mensagem da exceção deixa de ser a especificada). Eles sobreviveram porque a versão inicial dos testes usava `pytest.raises(ValueError, match=re.escape(MSG))`. `match` faz `re.search`, ou seja, aceita qualquer string que **contenha** o

texto esperado como substring — e o operador de mutação de string do mutmut envolve o literal original com marcadores `XX...XX`, produzindo exatamente uma superstring do texto esperado. O `match=` nunca detectaria esse tipo de mutação, não importa quantos casos de fronteira a suíte tivesse. Classificação: **lacuna da suíte** (defeito na asserção, não no sistema-base) — corrigida trocando `match=` por comparação exata (`str(exc_info.value) == MSG`).

Mutante 3 — manual, `frozen=True` → `frozen=False` (mutante equivalente)

```
- @dataclass(frozen=True)
+ @dataclass(frozen=False)
class EvaluationResult:
```

Este mutante não é gerado automaticamente pelo mutmut (está fora de qualquer corpo de função), então foi introduzido manualmente para completar a análise pedida pelo enunciado. Justificativa de equivalência: o contrato observável que esta suíte testa — conforme o próprio enunciado define ("verificar não apenas o status retornado, mas também a lista de motivos") — é exclusivamente o par `(status, reasons)` devolvido por `evaluate_scholarship`. Nenhuma chamada à função, para nenhuma combinação de `age`/`gpa`/`attendance_rate`/`has_required_courses`/`disciplinary_recor` consegue distinguir a versão `frozen=True` da `frozen=False`: os valores retornados são idênticos em todos os casos. A única forma de diferenciá-los é tentar reatribuir um campo do objeto retornado (`result.status = ...`) — algo que não faz parte do comportamento especificado, e que deliberadamente não testamos (ver nota no arquivo de testes, seção 8). Confirmamos experimentalmente: rodando a suíte final contra este mutante manual, todos os 29 testes passam (`mutation_reports/manual_equivalent_mutant/pytest_run.txt`) — ele de fato sobrevive, como esperado de um mutante equivalente.

5.3 Testes adicionados para matar as lacunas

As quatro asserções de exceção (`test_gpa_below_zero_raises_value_error`, `test_gpa_above_ten_raises_val` `test_attendance_below_zero_raises_value_error`, `test_attendance_above_hundred_raises_value_error`) passaram a comparar a mensagem **exatamente** (`assert str(exc_info.value) == MSG`), em vez de usar `match=`.

5.4 Score final

```
mutmut run   # após as correções acima
```

resultou em **84/84 mutantes mortos → score final = 100 %** (`mutation_reports/final/summary.json`). O mutante equivalente manual não entra nessa contagem porque não é gerado pela ferramenta (ver §5.1/§5.2).

	Score	Sobreviventes
Inicial	82/84 = 97,62 %	2 (ambos lacunas de asserção)
Final	84/84 = 100 %	0 (automáticos); 1 equivalente identificado manualmente

6. Análise de adequação

Cobertura vs. mutação. Medimos cobertura de linha/branch com `pytest-cov`: a suíte final atinge 100% de cobertura de linhas e branches em `evaluate_scholarship` e `_validate_inputs` (as únicas linhas não cobertas, 96% do arquivo, são o bloco `if __name__ == "__main__":`, corretamente inatingível durante os testes). É importante notar que **essa cobertura de 100% já existia na primeira versão da suíte** — antes de corrigirmos os dois `match=` — e mesmo assim 2 mutantes sobreviveram. Isso ilustra exatamente por que a cobertura de código não é suficiente: ela garante que cada branch foi *executado* ao menos uma vez, mas nada diz sobre a *força da asserção* usada para verificar o resultado daquela execução. Cobertura e mutação, nesse caso,

divergiram: 100% de cobertura desde o início, mas só 97,6% de mutação — a lacuna só apareceu porque a mutação de fato alterou o valor observável e o teste correspondente não percebeu.

Limitações da suíte. (1) Os limiares numéricos usados nos testes de fronteira (16, 17, 6.0, 7.0, 75.0, 80.0, 0–10, 0–100) foram extraídos do próprio código-fonte, não de um documento de requisitos independente. Se o sistema-base tiver um "bug intencional" que trocou, por exemplo, o limite de reprovação de frequência de 75% para 70%, nossa suíte (derivada da leitura do código) reproduziria e validaria esse valor errado como se fosse correto — nem cobertura nem mutação detectam esse tipo de defeito, porque ambas comparam o código com ele mesmo (mutantes) ou com testes derivados dele, nunca com uma especificação externa. (2) A suíte não testa propriedades não-funcionais (desempenho, concorrência) nem a ausência de regras — se uma sexta regra de negócio fosse esquecida na implementação, mutação e cobertura não teriam como apontar isso, pois só analisam o código que existe. (3) Tipos de entrada não são validados por tipo (ex.: `age` como `float` ou `str`) — o enunciado pede "entradas inválidas" no sentido de fora do domínio numérico esperado, que cobrimos; não testamos entradas de tipo incorreto porque a assinatura da função (tipada) e o escopo do enunciado (classes de equivalência de domínio, não de tipo) não pediam isso.

Considerando esses pontos, avaliamos a suíte como **adequada para o escopo definido pelo enunciado**: ela deriva sistematicamente da especificação (classes de equivalência e fronteiras) e da estrutura do código (todas as decisões e o caminho de mascaramento rejeição/revisão), atinge 100% de cobertura de branch e 100% de score de mutação (à parte do mutante equivalente, corretamente identificado e não perseguido), e verifica tanto `status` quanto `reasons`. Ela não é, e não pretende ser, uma prova de ausência de defeitos de especificação.

7. Respostas às perguntas do enunciado

1. Quais classes de equivalência foram identificadas? Ver §2.1 — 3 classes por critério numérico (idade, GPA, frequência), mais a classe de entrada inválida de cada um, mais 2 classes booleanas (cursos, disciplinar), mais as classes combinadas de decisão final (aprovação, rejeição por motivo N, revisão, e a classe de mascaramento rejeição-sobre-revisão).

2. Quais valores limite foram considerados? Ver §2.2 — os dois lados de cada fronteira de decisão (15/16, 17/18, 5.9/6.0, 6.9/7.0, 74.9/75.0, 79.9/80.0) e os extremos válidos/inválidos da validação de entrada (0.0/-0.1 e 10.0/10.1 para GPA; 0.0/-0.1 e 100.0/100.1 para frequência).

3. Quais decisões importantes do código foram testadas? Todas as 10 decisões (`if`/`elif`) das 5 regras de negócio e da validação de entrada, mais os 3 caminhos da decisão final e o caminho de mascaramento — ver tabela em §3.

4. Qual foi o score de mutação inicial e final? O que explica a diferença? Inicial 97,62% (82/84), final 100% (84/84). A diferença veio de uma única causa raiz: duas asserções de exceção usavam `pytest.raises(match=...)`, que faz busca de substring por regex em vez de igualdade exata, e por isso não detectava um mutante que envolvia a mensagem original com texto extra. Trocar por comparação exata da string eliminou os dois sobreviventes automáticos.

5. A cobertura de código e o score de mutação apontaram para as mesmas lacunas? Se divergiram, por quê? Divergiram: a suíte já tinha 100% de cobertura de linha/branch desde a primeira versão, mas só 97,6% de score de mutação. A cobertura mede execução; a mutação mede se o *resultado* daquela execução foi verificado com precisão suficiente. Neste caso a lacuna estava na asserção (uso de `match=`), não na falta de um caminho de execução — algo que cobertura, por definição, não consegue enxergar.

6. Quais mutantes você classificou como equivalentes, e com base em que argumento? Um: `@dataclass(frozen=True)` → `frozen=False` em `EvaluationResult` (introduzido manualmente, pois o mutmut não muta código fora de corpos de função). Argumento: o contrato observável testado pela suíte é `(status, reasons)`; nenhuma entrada válida ou inválida produz uma diferença observável nesse contrato entre as duas versões, porque a suíte nunca tenta mutar o objeto retor-

nado (isso está fora do escopo funcional especificado no enunciado). Ver §5.2 para a verificação experimental.

7. A suíte pode ser considerada adequada? Por quê? Sim, dentro do escopo do enunciado: deriva de técnicas funcionais e estruturais, cobre 100% dos branches relevantes, mata 100% dos mutantes que a ferramenta consegue gerar (à parte do único mutante equivalente identificado e justificado), e verifica `status` e `reasons` em todos os casos, incluindo o caso sutil de mascaramento. Ver §6 para as limitações reconhecidas.

8. Que tipos de defeito nenhuma das duas métricas seria capaz de revelar? Defeitos de especificação/requisito — quando o código implementa consistentemente uma regra errada (ex.: um limiar numérico incorreto, ou uma condição de fronteira invertida) e essa mesma regra é usada tanto para escrever o código quanto para derivar os testes, cobertura e mutação não têm como flagrar a discrepância, pois ambas comparam o código apenas consigo mesmo. Da mesma forma, nenhuma das duas métricas detecta **ausência** de código — uma regra de negócio inteira que deveria existir mas nunca foi implementada não gera nenhum mutante e nenhuma lacuna de cobertura, porque não há linha nenhuma para mutar ou cobrir. Por fim, ambas são cegas a requisitos não-funcionais (desempenho, segurança, concorrência, usabilidade) e a defeitos de integração com o restante do sistema.